Next-step build plan: slime on SageMaker HyperPod + AgentCore (and our own EC2) as one agentic-RL system

Status: **plan** (2026-10-12). Nothing here is built yet. The demo itself (`demo_terminalbench_kernelbench`) changes only once each gate below has passed.

Goal

Show the complete end-to-end setup for agentic RL on a large model:

- **SageMaker HyperPod runs everything that needs the policy's GPUs.** That's slime's trainer, its inference engines, the rollout buffer and the checkpoints, on 2 x `ml.g6e.48xlarge`: one node trains, the other runs several SGLang engines at once.
- **The sandboxes run outside the cluster, on the hybrid tier we already have:**

Substrate	Who manages it	Runs	Status
AgentCore microVM Runtimes	AgentCore	single-container Terminal-Bench tasks	in the demo
AgentCore Instance (g6 capacity provider)	AgentCore	KernelBench, one managed instance per episode	in the demo (default)
m5 EC2 warm pool	us	compose / x86-only / > 2 GB Terminal-Bench tasks	in the demo
g6 EC2 warm pool (plain EC2 GPUs)	us	KernelBench, one episode per GPU slot	**proven 2026-10-12** (`ad4e870`), opt-in

The message for the deck: HyperPod scales the model, AgentCore scales the environments, and our own EC2 fills the gaps (compose tasks, warm GPU slots). Everything above the sandbox is the same code, whichever substrate an episode lands on.

Where we are (the gate the user set)

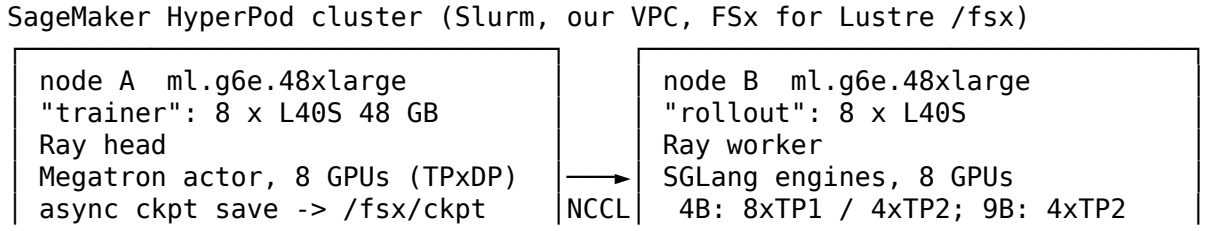
> "Only update the demo once the GPU instance not controlled by AgentCore is tested end to end."

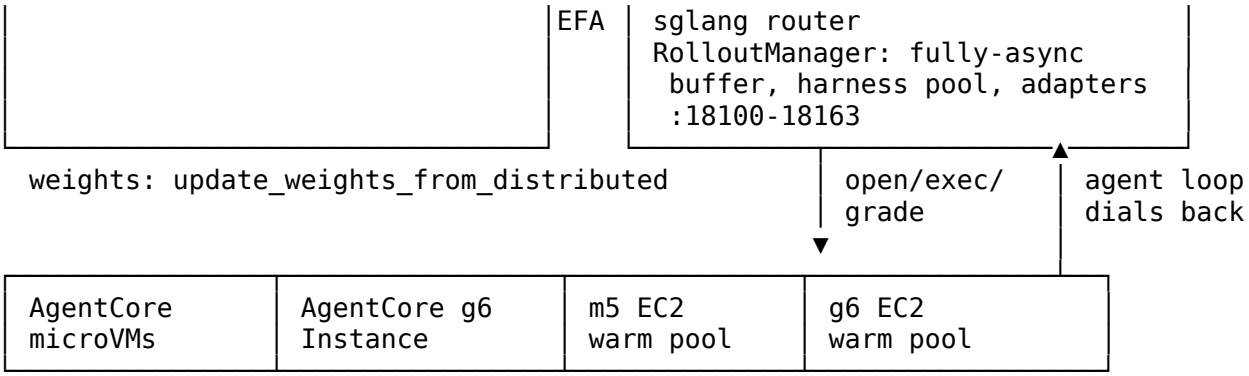
Gate G0: plain-EC2 GPU pool proven end to end. PASSED 2026-10-12. Evidence: `logs/kb-g6ec2-10step_20261012_091850` (KernelBench only, qwen3-4B):

- 10/10 steps plus the eval, 381 episodes, 0 infra errors;
- open p50 1.35 s, versus 133 s on the AgentCore g6;
- 95 % of opens reused a warm instance;
- every instance was a g5.xlarge, because g6 was out of capacity in all 4 AZs.

G0 proved KernelBench **alone** on the EC2 pool. The HyperPod demo runs all four substrates at once, so one more gate comes first, on the current single box (P1).

Target layout





- **Trainer (node A):** all 8 GPUs go to Megatron, versus 4 today. The 9B, which runs today at TP=4 with optimizer offload, gets 2x the GPUs (TP=4 x DP=2, or TP=8 without offload).
- **Inference (node B):** all 8 GPUs go to SGLang, versus 4 today. That gives 2-4x the engines today's `ROLLOUT_GPUS=4` allows, behind slime's router plus our per-episode engine pinning.
- **Rollout buffer:** slime's fully-async buffer lives in the RolloutManager process, and its depth follows `CONC`. No change, but the manager must end up on node B (see "slime on 2 nodes" below).
- **Model checkpoint:** saves (`--async-save`, persistent worker) and reloads go to **FSx for Lustre**, not EBS:
  - `/fsx/ckpt` for checkpoints. A 9B save is about 200 GB, which was slow on EBS (memory: qwen35-9b-on-l40s).
  - `/fsx/models` for the HF weights and torch_dist conversions.
  - `/fsx/logs` for the logs that `metrics.py` reads.
- **Weight sync:** crosses nodes now, from Megatron to the engines over NCCL. The g6e.48xlarge supports EFA (to be confirmed in P2). We'll measure the sync time per step. L40S has no NVLink, so it is already PCIe inside a node today.

Phases

P1: all four substrates in one run, on the current box (gate G1)

No HyperPod yet. This is the last change to the demo before HyperPod, and it is the "use the GPU instance not controlled by AgentCore" update the gate allows.

1. **KernelBench split routing.** Keep level 1 on the AgentCore g6 Instance, and route levels 2/3 (new rows) to the EC2 g6 pool. One image key per substrate (`kernelbench-gpu` and `kernelbench-gpu-ec2`), so `image_routing.py` stays a strict per-image map. New knob: `TB_KB_EC2_LEVELS=2,3` (`TB_KB_SUBSTRATE=ec2` keeps meaning "all of it on EC2").
2. **More KernelBench tasks** (agreed: after the substrate is proven, which it now is):
   - add levels 2 and 3 to `prepare_data.sh`, using the same checker and nonce;
   - re-check that the reward (1 / 0.5 / 0) still gives a gradient at those levels.
3. **Mixed 10-step run** on the full set: microVM + m5 + AgentCore g6 + EC2 g6, `TB_KB_SUBSTRATE` split.
   - **Exit criteria:** 0 infra errors on every substrate. The `by substrate` table in `metrics.py` shows all four. Cleanup leaves no instance behind.
   - **Pre-run:** start the m5 host first (it is stopped between runs). Never two `run.sh` at once.
4. **A/B on the same data:** KernelBench on AgentCore g6 vs EC2 g6. Measure open time, start wait, step time and $/episode. This fills the deck's `fr:gpuopts` EC2 column with like-for-like numbers.

P2: the HyperPod cluster (infra only)

New folder `deploy/hyperpod/`, with the same idempotent `up / status / down` style as `m5_host.py`:

- **Cluster:** `create-cluster` with the Slurm orchestrator. Instance groups:
  - `controller`: ml.m5.xlarge x1;
  - `trainer`: ml.g6e.48xlarge x1;
  - `rollout`: ml.g6e.48xlarge x1.

Lifecycle scripts install enroot + pyxis, mount FSx and pull the trainer image from ECR. The image is the committed `slimerl/slime:agentcore_p4-20261006` snapshot. **Never rebuild it from `:latest`.** *Decision to confirm:* Slurm vs EKS. Slurm is fewer moving parts for one Ray job on 2 nodes. EKS + KubeRay only pays off if the cluster is shared.

- **Networking:** the cluster's `VpcConfig` goes in the same default VPC, in the NAT-egress subnet of one AZ (`acs-egress-<az>`; both nodes in one AZ for EFA). One cluster SG:
  - self-referencing (Ray, NCCL/EFA);
  - in from the AgentCore runtime SG, the m5 SG and the g6ec2 SG on 18100-18163 (adapter dial-back);
  - out to the m5/g6 agents on 18300 plus the clients' published ports 32768-60999.

  `agentcore_deploy.trainer_sgs()` already honours `AGENTCORE_TRAINER_SG`. Set it to the cluster SG, so `open_dialback()` and `m5_host._ensure_sg()` write their rules against the cluster instead of this box.
- **IAM:** the cluster's instance role needs what this box's role does today:
  - `bedrock-agentcore:InvokeAgentRuntime` / `StopRuntimeSession` on `acs_*` Runtimes;
  - `ec2:RunInstances` / `TerminateInstances` / `Describe*` and `CreateTags`, tag-scoped to the m5/g6ec2 names;
  - `iam:PassRole` on `acs-tbh-m5-episode`;
  - ECR pull, and FSx.
- **Quotas:** SageMaker "ml.g6e.48xlarge for cluster usage" (request 2), plus FSx. Check before P2 starts: this is the likely long pole.
- **Image builds** stay on the current box (docker + buildx). HyperPod nodes only run.
- **Exit criteria:**
  - both nodes healthy, with `nvidia-smi` showing 8 GPUs each;
  - `nccl-tests all_reduce` across the 2 nodes over EFA, bandwidth recorded;
  - a node can `InvokeAgentRuntime` on a microVM Runtime;
  - a node can `RunInstances` a g6ec2 instance and reach its agent;
  - a microVM session dials an adapter port on node B.

P3: slime on 2 nodes (4B, microVM tasks first)

Code changes. The single-box layout stays the default; the new knob is `PLACEMENT=hyperpod`:

- **`run.sh`:**
  - start Ray as head on node A and join node B (`srun` per node; `MASTER_ADDR` = node A);
  - `--num-gpus 8` per node;
  - `ACTOR_GPUS=8`, `ROLLOUT_GPUS=8`, `--actor-num-nodes 1 --actor-num-gpus-per-node 8`.
- **Trainer placement (slime sorts its single placement group by node IP):** the actor takes the first 8 bundles, so **the lower-IP node is the trainer**. The launch script reads both IPs and assigns the roles that way (no slime patch). It asserts, from slime's bundle log line, that all 8 actor bundles sit on one node.
- **Adapter host:** slime's `RolloutManager` has no node pin (`num_cpus=1`), and the harness pool plus the adapters run inside it.
  - Change `in_session_harness` / `adapter_for` so that an unset `ADAPTER_PUBLIC_HOST` defaults to **the IP of the node the manager actually landed on** (`ray.util.get_node_ip_address()`).
  - The cluster SG admits the dial-back on both nodes, so wherever the manager lands works.
  - Record the chosen IP on a `[demo]` line.
- **`steps_common.sh`:** `in_container` gains a Slurm/pyxis path (`srun --container-image ...`) next to `docker exec`.
- **Smoke:** `NR=3`, microVM tasks only, 4B.
- **Exit criteria:**
  - 3 steps pass;
  - the weight-sync time per step is logged (new `perf/weight_sync_s` from slime's update timing);
  - the engines on node B serve all episodes (from `router.csv`).

P4: full hybrid on HyperPod + the larger model

1. **4B, full hybrid, 10 steps** (the P1 run, moved to HyperPod), with all four substrates. Compare with P1: step time, rollout throughput (episodes/min), weight-sync share.
2. **9B (`MODEL=qwen3.5-9B`):**
   - trainer TP=4 x DP=2, or TP=8 without optimizer offload: pick by memory, with the `MAX_TOKENS_PER_GPU` packing limit from the 2026-10-11 OOM in mind;
   - engines 4 x TP2 on node B;
   - checkpoints to `/fsx`.

   Exit criteria: 10 steps plus the eval, with save time on FSx vs EBS recorded.
3. **Stretch, "large model":** a 14B-32B class model needs more trainer memory than one 8 x 48 GB node (32B: bf16 weights 64 GB + fp32 grads + Adam). That means more `trainer` nodes in the instance group, with HyperPod adding nodes and the same code. This phase sizes it only; no build until P4.2 passes.

P5: resilience (the HyperPod selling point)

- **Checkpoint/resume on FSx:** kill the job mid-run, resubmit with `--load`, and the run continues from the last save.
- **HyperPod auto-resume:** run with Slurm `--auto-resume=1`, then simulate a faulty node, either by replacing a node or by the health-check path. Show that the job restarts from the FSx checkpoint and that the sandbox tiers clean up through their existing backstops:
  - AgentCore idle timeout;
  - the m5/g6 dead-man switch (`TB_M5_IDLE_SHUTDOWN_S`);
  - `gc` at pool start.

P6: demo + docs + deck (only after P4.1 passes)

- **Demo:** stays ONE demo (memory: demo-inventory). HyperPod becomes a placement variant of `demo_terminalbench_kernelbench`: `PLACEMENT=local` (default, today's box) or `PLACEMENT=hyperpod`. Add a README section with use case / why / design (memory: demo-docs-use-case-why-design), plus measured numbers.
- **Deck:** one architecture slide (the diagram above), one results slide (P1 vs P4 throughput, the open-time table, $/step), and the `fr:gpuopts` EC2 column filled from the P1 A/B.

Risks and open decisions

Risk / decision	Mitigation / default
ml.g6e.48xlarge cluster quota or capacity	Request in week 1. Fallback: ml.g6e.24xlarge x2 per role (4 GPUs each), which doubles the node count but keeps the same code
g6 capacity for the EC2 pool (starved in all AZs on 2026-10-12)	g5 fallback already proven (the user is fine with g5); ODCR / Capacity Block knob exists but is untested
Cross-node weight sync too slow	Measure in P3. If it dominates, sync less often (fully-async already tolerates staleness) or bucket the update
RolloutManager lands on the trainer node	The adapter host follows it (P3), so it works either way; its CPU load on node A is small
Slurm vs EKS	Slurm, unless a shared EKS cluster is a requirement
Cost	2 x g6e.48xlarge at roughly $30/h each on EC2 list price (the SageMaker ml. rate is higher; check). Run only for measured runs, and down the cluster between phases

Rules carried over (memory)

- Never two `run.sh` at once. Never edit `run.sh` or step scripts while a run is going.
- Start the m5 host before a Terminal-Bench run (it is stopped when idle).
- Never rebuild the trainer image from `:latest`.
- Every demo change: use case / why / design in the README, and measured numbers only.